

Reviewer Pack — Minimal Representation Rank of Formal Languages and Circuit ...

Entry fb585f05c28c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 17:06:01 UTC

Minimal Representation Rank of Formal Languages and Circuit Monotone Width Inequality

Entry ID: fb585f05c28c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 17:06:01 UTC

1. Conjecture

Field A (mathematical branch): Formal Language Theory (Representation Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every formal language L represented by a finite automaton with n states, the minimal representation rank of L , denoted as $r(L)$, is linearly related to its circuit monotone width, such that $r(L) = O(w_m(L))$ and $w_m(L) = \Theta(r(L))$.

Rationale (proposer's reasoning):

Formal language representation theory provides a structured way to encode languages, which can potentially reveal structural information about the complexity of computing them. Circuit monotone width is a measure of the expressive power of circuits, and a connection between these two areas could expose new insights into circuit complexity.

Taxonomy category: Formal_Language_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a804ccba971af94d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture on minimal representation rank and circuit monotone width, we will consider the conjecture supported if the Pearson correlation coefficient between $r(L)$ and $w_m(L)$ is ≥ 0.7 for at least 80% of the seeds ($n=40$), and falsified if this condition is not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - formal language theory AND representation theory AND circuit monotone width - automaton state count AND representation rank AND circuit complexity - minimal representation rank AND $\theta(\text{circuit monotone width})$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def tseitin_formula(n):
    if n <= 1:
        return []

    variables = [f'x{i}' for i in range(1, n+1)]
    clauses = []

    # Base case: x1 v ¬x2
    clauses.append([variables[0], -variables[1]])

    for i in range(2, n):
        new_var = f'x{i+1}'
        clauses.append([-variables[i-1], variables[i], -new_var])
        clauses.append([variables[i-1], -variables[i], new_var])
        clauses.append([-variables[i], -new_var])
        clauses.append([new_var])

    return clauses

def minimal_representation_rank(n):
    if n <= 1:
        return 0

    # Simplified rank calculation for demonstration
    return n - 1

def circuit_monotone_width(clauses):
    # Placeholder function to simulate monotone width calculation
    # In practice, this would involve a DPLL solver or other method
    return len(clauses)
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    metric_name = "Pearson correlation coefficient"
    instances_tested = 0
    n_max = 0
    r_values = []
    w_m_values = []

    for n in [5, 10, 15, 20, 30, 40]:
        if n > n_max:
            n_max = n

        instances_tested += 1
        clauses = tseitin_formula(n)

        if not clauses:
            continue

        rank = minimal_representation_rank(n)
        w_m = circuit_monotone_width(clauses)

        r_values.append(rank)
        w_m_values.append(w_m)

    if len(r_values) < 30:
        return {
            "metric_name": metric_name,
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    mean_r = sum(r_values) / len(r_values)
    mean_w_m = sum(w_m_values) / len(w_m_values)
    correlation_coefficient = (sum((r - mean_r) * (w_m - mean_w_m) for r, w_m in zip(r_values, w_m_values)) /
                             math.sqrt(sum((r - mean_r)**2 for r in r_values) *
                                         sum((w_m - mean_w_m)**2 for w_m in w_m_values)))

    return {
        "metric_name": metric_name,
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": abs(correlation_coefficient) >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
    results.append(result)

mean_value = sum(r['metric_value'] for r in results if r['metric_value'] is not None) / len(results)
std_value = math.sqrt(sum((r['metric_value'] - mean_value)**2 for r in results if r['metric_value'] is not None) / len(results))
support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

if all(r['conjecture_holds'] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r['conjecture_holds'])
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_below_threshold\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47f93013.py", line 106, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47f93013.py", line 63, in run_trial
    clauses = tseitin_formula(n)
               ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47f93013.py", line 26, in tseitin_formula
    clauses.append([variables[0], -variables[1]])
                   ~~~~~^~~~~~
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required computations to evaluate the conjecture. | next: Investigate and fix the error in the test code to ensure it can run to completion and produce the necessary data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14670
2	preregistration	ollama_remote	glm4:latest	0	0	9274
3	novelty	ollama_remote	glm4:latest	0	0	8331
4	novelty	ollama_remote	glm4:latest	0	0	9300
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28649
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12331
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14445
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11502
9	judge	ollama_remote	glm4:latest	0	0	18306

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 126808 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/fb585f05c28c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fb585f05c28c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fb585f05c28c.tar.gz` (if generated)